

UNIVERSIDADE DE SÃO PAULO (USP) — ESALQ
CURSO DE ESPECIALIZAÇÃO EM ENGENHARIA DE SOFTWARE

Estudo Comparativo de Modelos de Concorrência em Workloads I/O-Bound

Análise Estatística Local e Relatório Científico Preliminar (Java 25 vs Go 1.25 vs Quarkus Native)

Autor:	Felippe Gustavo de Souza e Silva
Orientador:	Prof. Marcos Jardel Henriques
Data:	Junho de 2026
Ambiente:	Apple M4 (4 vCPU / 8 GB RAM) — Colima (vz driver)

1. Resumo Executivo

Este relatório apresenta os resultados preliminares dos testes comparativos de performance, consumo de memória RAM e comportamento sob carga de três diferentes runtimes de backend estruturados em um gateway de pagamentos com latência simulada de I/O de 200–500ms. O experimento foi projetado para avaliar cientificamente a eficiência de três paradigmas de concorrência distintos:

- **Java 25 (Virtual Threads - Loom):** Modelo de agendamento M:N (muitos para muitos) executando sobre a JVM com o coletor ZGC Generational.
- **Go 1.25 (Goroutines):** Modelo de concorrência nativo M:N (Goroutines multiplexadas no runtime do Go).
- **Quarkus Native (OS Threads):** Modelo de threads tradicionais do SO 1:1, compilado nativamente via GraalVM Mandrel para isolamento de overhead do runtime.

Garantia Científica de Isolamento de Cache

Como metodologia rigorosa do TCC, todas as execuções de testes foram precedidas pelo comando `redis-cli FLUSHALL` e utilizaram chaves de idempotência únicas por iteração (`k6-vu${__VU}-iter${__ITER}`). Isso garante 100% de isolamento, forçando toda requisição a passar pela persistência e chamada externa, sem hits espúrios de cache.

2. Resultados Consolidados por Cenário

Cenário A: Baseline (20 VUs concorrentes) — Carga Baixa

Backend	Latência Média	p95	p99	Throughput (RPS)	RAM Pico (RSS)	Erros
Java 25 VT	358.0 ms	491.9 ms	504.4 ms	24.5 req/s	789 MB	0.00%
Go 1.25	354.1 ms	489.4 ms	500.4 ms	24.6 req/s	32.4 MB	0.00%
Quarkus Native	356.6 ms	491.9 ms	502.7 ms	24.4 req/s	55.5 MB	0.00%

Cenário B: Stress (200 VUs concorrentes) — Ponto de Saturação

Backend	Latência Média	p95	p99	Throughput (RPS)	RAM Pico (RSS)	Erros
Java 25 VT	353.4 ms	488.4 ms	500.5 ms	342.0 req/s	1378.7 MB	0.00%
Go 1.25	351.6 ms	486.3 ms	498.7 ms	343.4 req/s	49.7 MB	0.00%
Quarkus Native	353.5 ms	488.5 ms	500.4 ms	341.8 req/s	147.7 MB	0.00%

Cenário C: Spike (500 VUs concorrentes) — Carga Extrema

Backend	Latência Média	p95	p99	Throughput (RPS)	RAM Pico (RSS)	Erros
Java 25 VT	723.8 ms	917.8 ms	985.8 ms	362.3 req/s	1935.0 MB	0.00%
Go 1.25	353.6 ms	488.4 ms	504.8 ms	655.3 req/s	78.1 MB	0.14%
Quarkus Native	360.2 ms	495.2 ms	549.8 ms	646.3 req/s	463.8 MB	0.50%

3. Principais Conclusões Científicas

3.1. Equivalência até o Limite do Pool (Stress)

Com o dimensionamento de conexões simétrico de 200 conexões para o banco de dados, os três runtimes alcançaram um **empate estatístico completo** sob 200 VUs (Stress). Isso prova empiricamente que, em cargas I/O-bound moderadas, o modelo de threading (seja Threads de SO, Virtual Threads ou Goroutines) não limita o throughput da aplicação — todos entregaram cerca de 342 req/s com latência de resposta ditada exclusivamente pelo I/O externo.

3.2. Colapso do Java no Spike e Contenção do HikariCP

No cenário de Spike (500 VUs), a performance do Java degradou significativamente, caindo para 362 RPS, enquanto Go e Quarkus mantiveram a performance máxima estável (~650 RPS).

Lock Contention (Contenção de Bloqueio)

A degradação do Java não foi causada por thread pinning. A JVM foi monitorada com -Djdk.tracePinnedThreads=full e registrou **zero eventos de pinning** (graças ao JEP 491 entregue no Java 24 que resolveu o pinning de synchronized). A degradação deveu-se à contenção de lock interna do pool de conexões **HikariCP**, cuja arquitetura usa synchronized para serializar acessos críticos ao pool sob pressão extrema de 500 Virtual Threads ativas. O pool Agroal do Quarkus e o pgxpool do Go (baseado em canais lock-free) não sofreram com este gargalo.

3.3. Footprint de Memória (O Custo da JVM)

O teste confirma que o grande consumo de RAM do ecossistema Java está atrelado ao **runtime da JVM** e não ao código ou paradigma em si:

- **Java JVM:** Variou de 789 MB (Baseline) a 1.935 MB (Spike), devido ao heap do ZGC, metadados de classe (Metaspace) e code cache da JIT.
- **Quarkus Native (Compilação AOT):** Consumiu de 55 MB a 464 MB. Por ser compilado em binário nativo estático (GraalVM Mandrel), seu consumo é muito próximo ao do Go no baseline. Sob spike de 500 VUs (600 threads de SO configuradas), o consumo subiu para 464 MB devido ao custo fixo de alocação de stack das OS Threads (~512KB por thread de SO 1:1).
- **Go 1.25:** Manteve-se extremamente leve, consumindo de 32 MB (Baseline) a 78 MB (Spike), comprovando a eficiência das Goroutines (que inicializam com apenas ~2KB de stack dinâmica).

4. Próximos Passos: Validação no GKE (Cloud)

Os dados locais demonstraram com precisão o comportamento dos concorrentes. Para consolidação e defesa final da tese de engenharia de software na USP ESALQ, a próxima etapa consistirá na **validação destes resultados em ambiente Cloud real (GKE - Google Kubernetes Engine)**. Os manifests de deploy K8s e a esteira de benchmarks automatizados foram devidamente escritos e assinados com a chave GPG do autor para garantir rastreabilidade.

A bateria em nuvem testará a densidade prática de pods por node do Kubernetes, o custo por requisição (RPS/Dólar) e o comportamento do JIT da JVM em arquiteturas x86_64 nativas Linux.